

# Chapter 32 - IE Mon

IE Mon is the interactive monitor built into Intuition Engine. It runs in the same terminal as BASIC and gives you direct access to the bus, the active CPU's registers, breakpoints, watchpoints, a disassembler, a byte memory editor, a reverse-execution timeline, save and restore of memory ranges, and a small handful of more specialised tools.

You enter the monitor from BASIC by typing `MON` at the prompt and leave it by typing `x`. Inside the monitor, each line is a command followed by space-separated arguments. Numeric arguments default to hexadecimal; prefix them with `#` for decimal.

## 32.1 General conventions

| Convention       | Example   | Meaning                 |
|------------------|-----------|-------------------------|
| Address          | 1000      | Hexadecimal \$1000      |
| Address          | #4096     | Decimal                 |
| Address pair     | 1000 10FF | Inclusive range         |
| Register         | r0, pc, a | Active CPU's register   |
| Hex byte literal | 7E        | Hex byte for write/fill |

The monitor's prompt is the active CPU's short name followed by a greater-than sign: (`ie64`)>, (`6502`)>, (`z80`)>, etc.

## 32.2 Execution control

| Command        | Argument(s)            | Effect                                                 |
|----------------|------------------------|--------------------------------------------------------|
| <code>g</code> | [ <code>addr</code> ]  | Resume execution from current PC or <code>addr</code>  |
| <code>s</code> | [ <code>count</code> ] | Step: execute one instruction (or <code>count</code> ) |
| <code>u</code> | <code>addr</code>      | Run until: resume, stop one-shot at <code>addr</code>  |
| <code>x</code> |                        | Exit the monitor (return to BASIC)                     |

`s` always re-prints the registers after the step. `g` returns to the monitor on the next breakpoint, watchpoint, or `Ctrl-C`.

## 32.3 Registers and disassembly

| Command           | Argument(s)                                  | Effect                                             |
|-------------------|----------------------------------------------|----------------------------------------------------|
| <code>r</code>    | [ <code>reg</code> ] [ <code>value</code> ]  | Show all registers, or set one register            |
| <code>d</code>    | [ <code>addr</code> ] [ <code>count</code> ] | Disassemble at <code>addr</code>                   |
| <code>list</code> | [ <code>addr</code> ] [ <code>count</code> ] | List annotated lines when line data is loaded      |
| <code>cpu</code>  | [ <code>name</code> ]                        | Show the active CPU or switch to <code>name</code> |

The argument to `cpu` is `ie64`, `ie32`, `6502`, `z80`, `m68k`, or `x86`.

## 32.4 Memory inspection and editing

| Command | Argument(s)          | Effect                                  |
|---------|----------------------|-----------------------------------------|
| m       | addr [count]         | Memory dump in hex + ASCII              |
| w       | addr byte [byte...]  | Write bytes at addr                     |
| e       | addr                 | Enter interactive hex editor mode       |
| f       | start end byte       | Fill a range with one byte              |
| h       | start end pattern... | Hunt: search a range for a byte pattern |
| c       | start end dest       | Compare a range against dest            |
| t       | start end dest       | Transfer a range to dest                |

w is a one-line byte writer. Use e addr when you want interactive byte editing. Byte arguments should be 00 to FF; larger values are stored as their low byte.

There is no mnemonic-entry command. To enter machine code, write the bytes with w and confirm them with d.

### 32.4.1 Byte-entry audio workflow

This is the standard machine-language programming loop in IE Mon:

```
(6502)> w 1000 A9 01 8D 00 F8 A9 00 8D 08 D2 A9 79 8D 00 D2 A9
(6502)> w 1010 AF 8D 01 D2 4C 14 10
(6502)> d 1000 9
1000: A9 01          LDA #$01
1002: 8D 00 F8      STA $F800
1005: A9 00          LDA #$00
1007: 8D 08 D2      STA $D208
100A: A9 79          LDA #$79
100C: 8D 00 D2      STA $D200
100F: A9 AF          LDA #$AF
1011: 8D 01 D2      STA $D201
T 1014: 4C 14 10    JMP $1014
(6502)> r pc 1000
(6502)> b 1014
(6502)> g
(6502)> m D200 2
D200: 79 AF 00 00 00 00 00 00 00 00 C0 00 00 00 00 00  y.....
D210: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ....
(6502)> bc 1014
```

The leading prefix in each disassembly line tells you what the monitor thinks about that address: two spaces for an ordinary instruction, T for a branch target that another line in the window references (the self-loop at \$1014 here), > for the current PC, and \* for an active breakpoint. The d count argument is parsed as hexadecimal by default (so d 1000 9 gives nine lines and d 1000 10 would give sixteen); prefix the count with # for decimal. The m count argument is decimal by default and counts *rows of sixteen bytes*, not bytes. m D200 2 shows two sixteen-byte rows starting at \$D200, and the values written by the program appear in the first few columns of the first row.

The w line enters bytes. The d line proves those bytes decode as the intended instructions. The breakpoint stops execution before the final self-loop, and the memory dump proves the POKEY frequency and control bytes were written. You should

hear the tone while the audio engine is enabled. If the disassembly does not match the chapter transcript, fix the byte listing before running it.

## 32.4.2 Byte-entry graphics workflow

The same loop works for visible hardware. This 6502 example uses the ULA card: it sets the border, enables the ULA source, writes an 8-byte bitmap motif through the ULA latch/data port, writes one attribute byte, and then loops. It is deliberately small, but it does more than poke a colour register: it teaches the latched VRAM entry path used by larger ULA programs.

```
(6502)> w 1100 A9 05 8D 00 D8 A9 05 8D 04 D8 A9 00 8D 0C D8 A9
(6502)> w 1110 00 8D 10 D8 A9 FF 8D 14 D8 A9 81 8D 14 D8 A9 BD
(6502)> w 1120 8D 14 D8 A9 A5 8D 14 D8 A9 A5 8D 14 D8 A9 BD 8D
(6502)> w 1130 14 D8 A9 81 8D 14 D8 A9 FF 8D 14 D8 A9 00 8D 0C
(6502)> w 1140 D8 A9 18 8D 10 D8 A9 46 8D 14 D8 4C 4B 11
(6502)> d 1100 #31
1100: A9 05          LDA #$05
1102: 8D 00 D8       STA $D800
1105: A9 05          LDA #$05
1107: 8D 04 D8       STA $D804
110A: A9 00          LDA #$00
110C: 8D 0C D8       STA $D80C
110F: A9 00          LDA #$00
1111: 8D 10 D8       STA $D810
1114: A9 FF          LDA #$FF
1116: 8D 14 D8       STA $D814
1119: A9 81          LDA #$81
111B: 8D 14 D8       STA $D814
111E: A9 BD          LDA #$BD
1120: 8D 14 D8       STA $D814
1123: A9 A5          LDA #$A5
1125: 8D 14 D8       STA $D814
1128: A9 A5          LDA #$A5
112A: 8D 14 D8       STA $D814
112D: A9 BD          LDA #$BD
112F: 8D 14 D8       STA $D814
1132: A9 81          LDA #$81
1134: 8D 14 D8       STA $D814
1137: A9 FF          LDA #$FF
1139: 8D 14 D8       STA $D814
113C: A9 00          LDA #$00
113E: 8D 0C D8       STA $D80C
1141: A9 18          LDA #$18
1143: 8D 10 D8       STA $D810
1146: A9 46          LDA #$46
1148: 8D 14 D8       STA $D814
T 114B: 4C 4B 11     JMP $114B
(6502)> r pc 1100
(6502)> b 114B
(6502)> g
(6502)> m D800 1
D800: 05 00 00 00 05 00 00 00 01 00 00 00 02 00 00 00 .....
(6502)> bc 114B
```

The writes to \$D80C and \$D810 set the low and high halves of the ULA VRAM address latch. Each store to \$D814 writes one byte at the latched address and advances the latch. The first eight data bytes form the bitmap motif. The later latch

value \$1800 selects the attribute area, and \$46 gives the cell yellow ink on black paper. The d count of #31 is decimal (the # prefix forces decimal. Without it the count is parsed as hexadecimal, which is the default for d); the single m D800 1 shows the sixteen-byte row that starts at \$D800 so the writes to \$D800 (05) and \$D804 (05) appear in the first row. The \$D814 register is a write-only auto-incrementing latch port; reading it does not step through the bitmap data the program wrote, so the verification is the visible stripe on the ULA display, not a follow-on m D814 read. Try changing the first \$A5 byte to \$99; the middle of the motif changes without moving the attribute cell.

## 32.5 Breakpoints

| Command | Argument(s) | Effect                                        |
|---------|-------------|-----------------------------------------------|
| b       | addr        | Set a code breakpoint at addr                 |
| bc      | [addr   id] | Clear a breakpoint by address or by ID        |
| bl      |             | List all breakpoints                          |
| bfirst  |             | Break on the first hit only (set policy flag) |

Breakpoints are sticky: setting one a second time at the same address increments a hit-counter rather than creating a duplicate.

## 32.6 Watchpoints (memory)

The monitor distinguishes between **byte**, **word**, **dword**, and **qword** watchpoints, and between **read**, **write**, and **read/write** flavours. The mnemonic encoding is:

bpm <size> <mode>      where size = b/w/d/q and mode = r/w/(both)

| Command        | Effect                                     |
|----------------|--------------------------------------------|
| ww addr        | Word watchpoint, read/write (legacy alias) |
| wr addr        | Word watchpoint, read only                 |
| wrw addr       | Word watchpoint, read only (verbose alias) |
| bpmb r addr    | Byte read                                  |
| bpbw w addr    | Byte write                                 |
| bpbm r/w addr  | Byte read/write                            |
| bpmw r addr    | Word read                                  |
| bpmw w addr    | Word write                                 |
| bpmw r/w addr  | Word read/write                            |
| bpm d r addr   | Dword read                                 |
| bpm d w addr   | Dword write                                |
| bpm d r/w addr | Dword read/write                           |
| bpm q r addr   | Qword read                                 |
| bpm q w addr   | Qword write                                |

| Command        | Effect               |
|----------------|----------------------|
| bpmq addr      | Qword read/write     |
| wc [addr   id] | Clear a watchpoint   |
| wl             | List all watchpoints |

These are memory watchpoints: they do not write anything. The similar-looking ww/wr are the abbreviations sometimes confused with byte writers - they are not.

## 32.7 Reverse execution

The monitor records a per-instruction history of the active CPU's state. You can rewind, replay forward, and search the timeline.

| Command | Argument(s) | Effect                                                               |
|---------|-------------|----------------------------------------------------------------------|
| bs      | [count]     | Back-step: undo one (or count) instructions                          |
| rs      | [count]     | Same as bs                                                           |
| rg      |             | Reverse-continue: rewind until the previous breakpoint or watchpoint |
| rt      | addr        | Reverse-run-until: rewind until PC was addr                          |
| tl      |             | Timeline: dump the recent history                                    |
| history |             | Same as tl                                                           |

Be careful: rg is **reverse-continue**, not "register inspect".

## 32.8 Tracing

| Command   | Argument(s) | Effect                                            |
|-----------|-------------|---------------------------------------------------|
| trace     | on off name | Enable or disable instruction trace               |
| tracering | [size]      | Enable a ring-buffer trace of recent instructions |
| show      |             | Dump the ring trace                               |

trace is the named trace command. It is distinct from t (which is **transfer memory**, not trace).

## 32.9 CPU freeze and thaw

These commands stop and resume execution of a specific CPU without exiting the monitor.

| Command | Argument(s) | Effect                             |
|---------|-------------|------------------------------------|
| freeze  | cpuName   * | Freeze one CPU, or * for all       |
| thaw    | cpuName   * | Thaw one CPU, or * for all         |
| fa      |             | Freeze audio (mixer + all engines) |
| ta      |             | Thaw audio                         |

fa and ta are **audio-only** controls, not freeze-all aliases. To freeze every CPU at once, use freeze \*.

## 32.10 Save and restore

| Command | Argument(s)    | Effect                                   |
|---------|----------------|------------------------------------------|
| save    | start end name | Save a memory range                      |
| load    | name addr      | Load a memory range at <code>addr</code> |
| ss      | name           | Save full machine state                  |
| sl      | name           | Load full machine state                  |

save and load move memory ranges. ss and sl move the whole CPU + bus + device state. A saved state can be reloaded into a freshly reset system to resume from the same point.

## 32.11 Symbols, addresses, maps

| Command | Argument(s)   | Effect                                       |
|---------|---------------|----------------------------------------------|
| sym     | [name   addr] | Look up a symbol or address                  |
| map     |               | Show the active loaded symbol map            |
| addr    | addr          | Resolve <code>addr</code> to a symbol if any |

## 32.12 Bus, MMIO, page-level access

| Command   | Argument(s)             | Effect                                  |
|-----------|-------------------------|-----------------------------------------|
| io        | [device   addr]         | Show MMIO region map or a specific page |
| pg        | add start end mode      | Page guard: trap on access to a range   |
| pg        | list   clear            | List or clear page guards               |
| accesslog | on off show [count]     | Control per-page access logging         |
| who       | read wrote fetched addr | Report the last matching access         |

## 32.13 Backtrace

| Command | Argument(s) | Effect                              |
|---------|-------------|-------------------------------------|
| bt      |             | Stack backtrace from the active CPU |

## 32.14 Command Results and Limits

Commands that cannot parse an address, register, CPU name, or byte value print an error line and leave the monitor active. d, m, r, io, bt, tl, wl, and bl inspect state. w, f, t, load, sl, register writes through r, and execution commands can change state.

d disassembles bytes; it never changes the program counter. g leaves the monitor until a breakpoint, watchpoint, manual break-in, or CPU stop brings control back. The reverse timeline is a recent-history tool, not permanent storage; use ss when you need a reloadable state.

## 32.15 Quick Reference Card

---

The single-letter and short commands you will use most:

|           |                        |            |                   |
|-----------|------------------------|------------|-------------------|
| r         | show registers         | bt         | backtrace         |
| d [a]     | disassemble            | s [n]      | step              |
| m [a]     | memory dump            | g [a]      | go                |
| w a b...  | write bytes            | u a        | run until         |
| b a       | breakpoint set         | bs [n]     | back-step         |
| bc [a id] | breakpoint clear       | rg         | reverse-continue  |
| bl        | breakpoint list        | rt a       | reverse-run-until |
| ww/wr a   | word watchpoint        | tl         | timeline          |
| wc [a id] | watchpoint clear       | x          | exit              |
| wl        | watchpoint list        | cpu n      | switch CPU        |
| freeze *  | freeze all CPUs        | thaw *     | thaw all          |
| fa / ta   | freeze / thaw audio    | save a b n | save range        |
| ss / sl n | save / load full state | load n a   | load range at a   |

## 32.16 What Comes Next

---

Chapter 33 covers IE Script, a command language that drives the monitor from stored command text rather than from a terminal. IE Script is what you use to automate session setup, breakpoint actions, data dumps after a fault, and similar batch debugging tasks.